

NEXARETAIL ORDERS ANALYSIS

Importing required libraries

```
import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

# Display plots inside notebook
%matplotlib inline

# Improve chart appearance
sns.set_style("whitegrid")

print("Libraries Imported Successfully!")

Libraries Imported Successfully!
```

Load dataset

```
df = pd.read_csv("P3_Dataset.csv")

df.shape

(11875, 12)
```

Dataset Information

```
df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 11875 entries, 0 to 11874
Data columns (total 12 columns):
#   Column          Non-Null Count  Dtype  
---  -
0   order_date      11875 non-null  object 
1   store_name      11875 non-null  object 
2   store_type      11875 non-null  object 
3   city            11875 non-null  object 
4   state           11875 non-null  object 
5   product_name    11875 non-null  object
```

```
6   category      11875 non-null object
7   quantity      11875 non-null int64
8   unit_price     11875 non-null int64
9   total_amount   11875 non-null int64
10  channel        11875 non-null object
11  return_reason   4443 non-null object
dtypes: int64(3), object(9)
memory usage: 1.1+ MB
```

Display column names

```
df.columns

Index(['order_date', 'store_name', 'store_type', 'city', 'state',
      'product_name', 'category', 'quantity', 'unit_price',
      'total_amount',
      'channel', 'return_reason'],
      dtype='object')
```

Number of rows and columns

```
print("Rows and Columns:", df.shape)

print("Rows:", df.shape[0])

print("Columns:", df.shape[1])

Rows and Columns: (11875, 12)
Rows: 11875
Columns: 12
```

Missing values

```
df.isnull().sum()

order_date      0
store_name      0
store_type      0
city            0
state           0
product_name    0
category        0
quantity        0
unit_price      0
total_amount    0
```

```
channel          0
return_reason    7432
dtype: int64
```

Missing value percentage

```
missing_percent = (df.isnull().sum() / len(df)) * 100
```

```
missing_percent
```

```
order_date      0.000000
store_name      0.000000
store_type      0.000000
city            0.000000
state           0.000000
product_name    0.000000
category        0.000000
quantity        0.000000
unit_price      0.000000
total_amount    0.000000
channel         0.000000
return_reason   62.585263
dtype: float64
```

Duplicate records

```
duplicates = df.duplicated().sum()

print("Duplicate Rows:", duplicates)

Duplicate Rows: 0
```

Summary statistics for numerical columns

```
df.describe()

{"summary": "{\n  \"name\": \"df\",\n  \"rows\": 8,\n  \"fields\": [\n    {\n      \"column\": \"quantity\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 4191.697539817623,\n        \"min\": 1.0,\n        \"max\": 11875.0,\n        \"num_unique_values\": 8,\n        \"samples\": [\n          20.43637894736842,\n          20.0,\n          11875.0\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"unit_price\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
```

```

4131.779343294806,\n          \"min\": 30.0,\n          \"max\": 11875.0,\n          \"num_unique_values\": 8,\n          \"samples\": [\n195.31621052631579,\n          110.0,\n          11875.0\n          ],\n          \"semantic_type\": \"\",\n          \"description\": \"\"\n        },\n        {\n          \"column\": \"total_amount\",\n          \"properties\": {\n            \"dtype\": \"number\",\n            \"std\": 7297.273309334575,\n            \"min\": 30.0,\n            \"max\": 22000.0,\n            \"num_unique_values\": 8,\n            \"samples\": [\n4012.9642105263156,\n            2100.0,\n            11875.0\n            ],\n            \"semantic_type\": \"\",\n            \"description\": \"\"\n          }\n        }\n      ],\n      \"type\": \"dataframe\"}

```

Summary of categorical columns

```
df.describe(include='object')
```

```

{"summary": "{\n  \"name\": \"df\",\n  \"rows\": 4,\n  \"fields\": [\n    {\n      \"column\": \"order_date\",\n      \"properties\": {\n        \"dtype\": \"date\",\n        \"min\": \"1970-01-01 00:00:00.000000061\",\n        \"max\": \"2023-10-01 00:00:00\",\n        \"num_unique_values\": 4,\n        \"samples\": [\n          \"61\",\n          \"11875\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"store_name\",\n      \"properties\": {\n        \"dtype\": \"string\",\n        \"num_unique_values\": 4,\n        \"samples\": [\n          40,\n          \"327\",\n          \"11875\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"store_type\",\n      \"properties\": {\n        \"dtype\": \"string\",\n        \"num_unique_values\": 4,\n        \"samples\": [\n          2,\n          \"8912\",\n          \"11875\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"city\",\n      \"properties\": {\n        \"dtype\": \"string\",\n        \"num_unique_values\": 4,\n        \"samples\": [\n          10,\n          \"1254\",\n          \"11875\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"state\",\n      \"properties\": {\n        \"dtype\": \"string\",\n        \"num_unique_values\": 4,\n        \"samples\": [\n          2,\n          \"7179\",\n          \"11875\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"product_name\",\n      \"properties\": {\n        \"dtype\": \"string\",\n        \"num_unique_values\": 4,\n        \"samples\": [\n          21,\n          \"610\",\n          \"11875\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"category\",\n      \"properties\": {\n        \"dtype\": \"string\",\n        \"num_unique_values\": 4,\n        \"samples\": [\n          2,\n          \"7179\",\n          \"11875\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    ]\n  }\n}

```

```
[\n      8,\n      \"1736\",,\n      \"11875\",,\n      \"semantic_type\": \"\",,\n      \"description\": \"\",,\n      },,\n      {\n      \"column\": \"channel\",,\n      \"properties\": {\n      \"dtype\": \"string\",,\n      \"num_unique_values\": 4,\n      \"samples\": [\n      \"8313\",,\n      \"11875\",,\n      ],,\n      \"semantic_type\": \"\",,\n      \"description\": \"\",,\n      },,\n      {\n      \"column\": \"return_reason\",,\n      \"properties\": {\n      \"dtype\": \"string\",,\n      \"num_unique_values\": 4,\n      \"samples\": [\n      \"3\",,\n      \"1507\",,\n      \"4443\",,\n      ],,\n      \"semantic_type\": \"\",,\n      \"description\": \"\",,\n      },,\n      ],,\n      ],,\n      \"type\": \"dataframe\"}
```

Convert order_date to datetime format

```
df['order_date'] = pd.to_datetime(df['order_date'])
```

UPDATED ORDER DATE COLUMN

```
df['order_date'].head()
```

```
0    2023-01-16
1    2023-01-28
2    2023-01-05
3    2023-01-13
4    2023-01-12
Name: order_date, dtype: object
```

Create Month Number

```
df['order_date'] = pd.to_datetime(df['order_date'])
df['Month'] = df['order_date'].dt.month
```

```
df[['order_date', 'Month']].head()
```

```
{\"summary\":{\n  \"name\": \"df[['order_date', 'Month']]\",,\n  \"rows\": 5,\n  \"fields\": [\n    {\n      \"column\":\n      \"order_date\",,\n      \"properties\": {\n      \"dtype\":\n      \"date\",,\n      \"min\": \"2023-01-05 00:00:00\",,\n      \"max\":\n      \"2023-01-28 00:00:00\",,\n      \"num_unique_values\": 5,\n      \"samples\": [\n      \"2023-01-28 00:00:00\",,\n      \"2023-01-12 00:00:00\",,\n      \"2023-01-05 00:00:00\",,\n      ],,\n      \"semantic_type\": \"\",,\n      \"description\": \"\",,\n      },,\n      {\n      \"column\": \"Month\",,\n      \"properties\": {\n      \"dtype\": \"int32\",,\n      \"num_unique_values\": 1,\n      \"samples\": [\n      \"1\",,\n      \"2\",,\n      \"3\",,\n      \"4\",,\n      \"5\",,\n      \"6\",,\n      \"7\",,\n      \"8\",,\n      \"9\",,\n      \"10\",,\n      \"11\",,\n      \"12\",,\n      ],,\n      \"semantic_type\": \"\",,\n      \"description\": \"\",,\n      },,\n      ],,\n      \"type\": \"dataframe\"}
```

```
\samples\": [\n          1\n        ],\n          \"semantic_type\":  
\\\"\", \n          \"description\": \"\\\"\\n          }\n        }\n      ]\n    }\", \"type\": \"dataframe\"}
```

Weekday

```
df['Weekday'] = df['order_date'].dt.day_name()

df[['order_date', 'Weekday']].head()

{"summary": "{\n  \"name\": \"df[['order_date', 'Weekday']]\", \n  \"rows\": 5, \n  \"fields\": [\n    {\n      \"column\": \"order_date\", \n      \"properties\": {\n        \"dtype\": \"date\", \n        \"min\": \"2023-01-05 00:00:00\", \n        \"max\": \"2023-01-28 00:00:00\", \n        \"num_unique_values\": 5, \n        \"samples\": [\n          \"2023-01-28 00:00:00\", \n          \"2023-01-12 00:00:00\", \n          \"2023-01-05 00:00:00\" \n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\" \n      } \n    }, \n    {\n      \"column\": \"Weekday\", \n      \"properties\": {\n        \"dtype\": \"string\", \n        \"num_unique_values\": 4, \n        \"samples\": [\n          \"Saturday\", \n          \"Friday\", \n          \"Monday\" \n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\" \n      } \n    } \n  ], \n  \"type\": \"dataframe\"}
```

Week Number

```
df['Week_Number'] = df['order_date'].dt.isocalendar().week

df[['order_date', 'Week_Number']].head()

{"summary": "{\n  \"name\": \"df[['order_date', 'Week_Number']]\", \n  \"rows\": 5, \n  \"fields\": [\n    {\n      \"column\": \"order_date\", \n      \"properties\": {\n        \"dtype\": \"date\", \n        \"min\": \"2023-01-05 00:00:00\", \n        \"max\": \"2023-01-28 00:00:00\", \n        \"num_unique_values\": 5, \n        \"samples\": [\n          \"2023-01-28 00:00:00\", \n          \"2023-01-12 00:00:00\", \n          \"2023-01-05 00:00:00\" \n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\" \n      } \n    }, \n    {\n      \"column\": \"Week_Number\", \n      \"properties\": {\n        \"dtype\": \"UInt32\", \n        \"num_unique_values\": 4, \n        \"samples\": [\n          2, \n          3 \n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\" \n      } \n    } \n  ], \n  \"type\": \"dataframe\" \n}
```

Create Return Status

```
import numpy as np
df['Return_Status'] = np.where(
    df['return_reason'].isnull(),
    'No Return',
    'Returned'
)

df[['return_reason', 'Return_Status']].head(10)

{"summary": "{\n  \"name\": \"df[['return_reason', 'Return_Status']]\", \n  \"rows\": 10, \n  \"fields\": [\n    {\n      \"column\": \"return_reason\", \n      \"properties\": {\n        \"dtype\": \"category\", \n        \"num_unique_values\": 1, \n        \"samples\": [\n          \"Late Delivery\" \n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\" \n      }, \n      \"column\": \"Return_Status\", \n      \"properties\": {\n        \"dtype\": \"category\", \n        \"num_unique_values\": 2, \n        \"samples\": [\n          \"Returned\" \n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\" \n      } \n    ] \n  }, \"type\": \"dataframe\"}
```

Verify new data types

```
df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 11875 entries, 0 to 11874
Data columns (total 14 columns):
#   Column                Non-Null Count  Dtype  
---  -
0   order_date            11875 non-null  datetime64[ns]
1   store_name            11875 non-null  object  
2   store_type            11875 non-null  object  
3   city                  11875 non-null  object  
4   state                 11875 non-null  object  
5   product_name          11875 non-null  object  
6   category              11875 non-null  object  
7   quantity              11875 non-null  int64  
8   unit_price            11875 non-null  int64  
9   total_amount          11875 non-null  int64  
10  channel               11875 non-null  object  
11  return_reason         4443 non-null   object  
12  Month                 11875 non-null  int32  
13  Return_Status         11875 non-null  object  
dtypes: datetime64[ns](1), int32(1), int64(3), object(9)
memory usage: 1.2+ MB
```

Display first five rows

```
df.head()
```

```

{"summary": "{\n  \"name\": \"df\",\n  \"rows\": 11875,\n  \"fields\": [\n    {\n      \"column\": \"order_date\",\n      \"properties\": {\n        \"dtype\": \"date\",\n        \"min\": \"2023-01-01 00:00:00\",\n        \"max\": \"2023-12-28 00:00:00\",\n        \"num_unique_values\": 336,\n        \"samples\": [\n          \"2023-03-17 00:00:00\",\n          \"2023-04-17 00:00:00\",\n          \"2023-11-23 00:00:00\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\",\n        \"column\": \"store_name\",\n        \"properties\": {\n          \"dtype\": \"category\",\n          \"num_unique_values\": 40,\n          \"samples\": [\n            \"Desai Provisions\",\n            \"Kulkarni Store\",\n            \"Rajput Provisions\"\n          ],\n          \"semantic_type\": \"\",\n          \"description\": \"\",\n          \"column\": \"store_type\",\n          \"properties\": {\n            \"dtype\": \"category\",\n            \"num_unique_values\": 2,\n            \"samples\": [\n              \"Supermarket\",\n              \"Kirana\"\n            ],\n            \"semantic_type\": \"\",\n            \"description\": \"\",\n            \"column\": \"city\",\n            \"properties\": {\n              \"dtype\": \"category\",\n              \"num_unique_values\": 10,\n              \"samples\": [\n                \"Solapur\",\n                \"Pune\"\n              ],\n              \"semantic_type\": \"\",\n              \"description\": \"\",\n              \"column\": \"state\",\n              \"properties\": {\n                \"dtype\": \"category\",\n                \"num_unique_values\": 2,\n                \"samples\": [\n                  \"Gujarat\",\n                  \"Maharashtra\"\n                ],\n                \"semantic_type\": \"\",\n                \"description\": \"\",\n                \"column\": \"product_name\",\n                \"properties\": {\n                  \"dtype\": \"category\",\n                  \"num_unique_values\": 21,\n                  \"samples\": [\n                    \"NexaGold Wheat Atta 10kg\",\n                    \"NexaClean Floor Cleaner 1L\"\n                  ],\n                  \"semantic_type\": \"\",\n                  \"description\": \"\",\n                  \"column\": \"category\",\n                  \"properties\": {\n                    \"dtype\": \"category\",\n                    \"num_unique_values\": 8,\n                    \"samples\": [\n                      \"Dal\",\n                      \"Spices\"\n                    ],\n                    \"semantic_type\": \"\",\n                    \"description\": \"\",\n                    \"column\": \"quantity\",\n                    \"properties\": {\n                      \"dtype\": \"number\",\n                      \"std\": 11,\n                      \"min\": 1,\n                      \"max\": 40,\n                      \"num_unique_values\": 40,\n                      \"samples\": [\n                        18,\n                        2\n                      ],\n                      \"semantic_type\": \"\",\n                      \"description\": \"\",\n                      \"column\": \"unit_price\",\n                      \"properties\": {\n                        \"dtype\": \"number\",\n                        \"std\": 169,\n                        \"min\": 30,\n                        \"max\": 550,\n                      }\n                    }\n                  }\n                }\n              }\n            }\n          }\n        }\n      }\n    }\n  ]\n}

```



```

\"num_unique_values\": 8,\n        \"samples\": [\n            50,\n            70,\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\",\n        \"column\": \"total_amount\",\n        \"properties\": {\n            \"dtype\": \"number\",\n            \"std\": 4596,\n            \"min\": 30,\n            \"max\": 22000,\n            \"num_unique_values\": 226,\n            \"samples\": [\n                1050,\n                2750,\n            ],\n            \"semantic_type\": \"\",\n            \"description\": \"\",\n            \"column\": \"channel\",\n            \"properties\": {\n                \"dtype\": \"category\",\n                \"num_unique_values\": 2,\n                \"samples\": [\n                    \"Online\",\n                    \"Offline\",\n                ],\n                \"semantic_type\": \"\",\n                \"description\": \"\",\n                \"column\": \"return_reason\",\n                \"properties\": {\n                    \"dtype\": \"category\",\n                    \"num_unique_values\": 3,\n                    \"samples\": [\n                        \"Late Delivery\",\n                        \"Damaged\",\n                    ],\n                    \"semantic_type\": \"\",\n                    \"description\": \"\",\n                    \"column\": \"Month\",\n                    \"properties\": {\n                        \"dtype\": \"int32\",\n                        \"num_unique_values\": 12,\n                        \"samples\": [\n                            11,\n                            10,\n                        ],\n                        \"semantic_type\": \"\",\n                        \"description\": \"\",\n                        \"column\": \"Return_Status\",\n                        \"properties\": {\n                            \"dtype\": \"category\",\n                            \"num_unique_values\": 2,\n                            \"samples\": [\n                                \"Returned\",\n                                \"No Return\",\n                            ],\n                            \"semantic_type\": \"\",\n                            \"description\": \"\"\n                        }\n                    }\n                }\n            }\n        ],\n    },\n    \"type\": \"dataframe\", \"variable_name\": \"df\"}

```

Revenue by City

```

city_sales = df.groupby('city')
['total_amount'].sum().sort_values(ascending=False)

plt.figure(figsize=(10,6))

sns.barplot(
    x=city_sales.values,
    y=city_sales.index,
    palette='viridis'
)

plt.title("Revenue by City")
plt.xlabel("Revenue (₹)")
plt.ylabel("City")

plt.show()

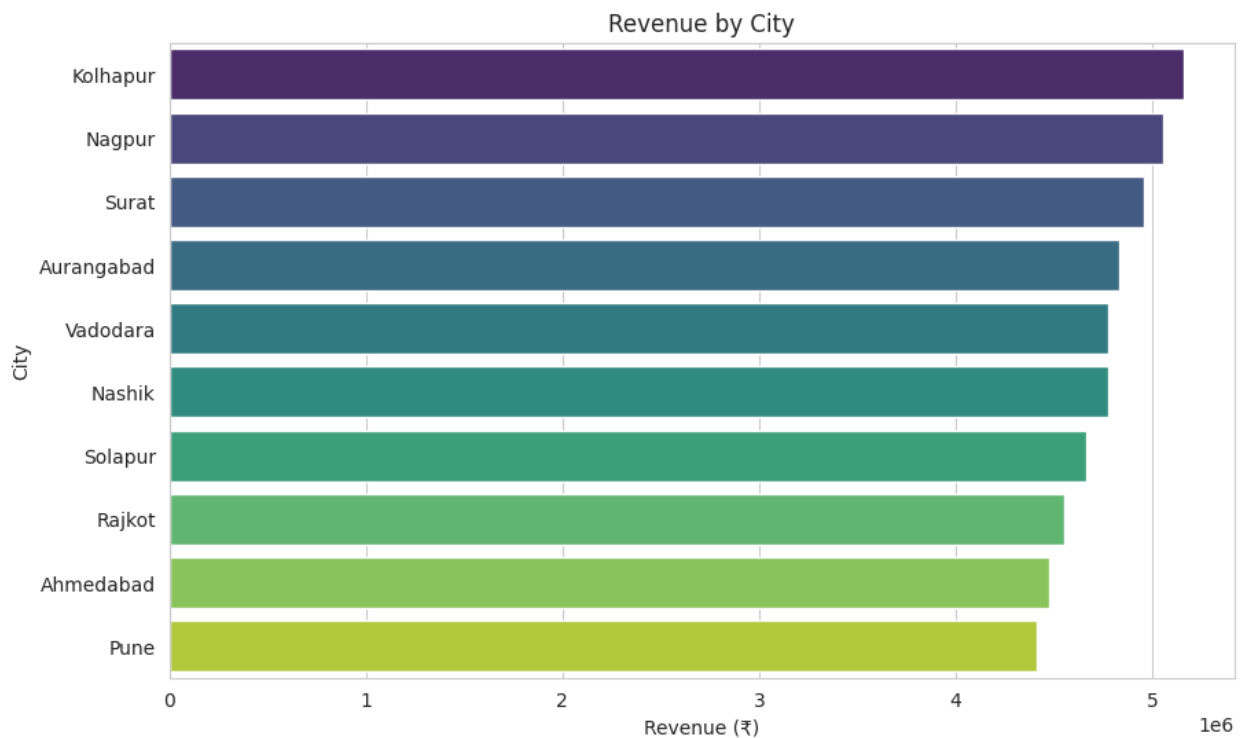
city_sales

```

```
/tmp/ipykernel_1610/1548148344.py:5: FutureWarning:
```

```
Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.
```

```
sns.barplot(
```



```
city
Kolhapur    5157800
Nagpur      5055540
Surat       4958680
Aurangabad  4831770
Vadodara    4778070
Nashik      4773550
Solapur     4663150
Rajkot      4552270
Ahmedabad   4472620
Pune        4410500
Name: total_amount, dtype: int64

store_sales = df.groupby('store_type')['total_amount'].sum()

plt.figure(figsize=(6,5))

sns.barplot(
    x=store_sales.index,
```

```

    y=store_sales.values,
    palette='Set2'
)

plt.title("Revenue by Store Type")
plt.xlabel("Store Type")
plt.ylabel("Revenue (₹)")

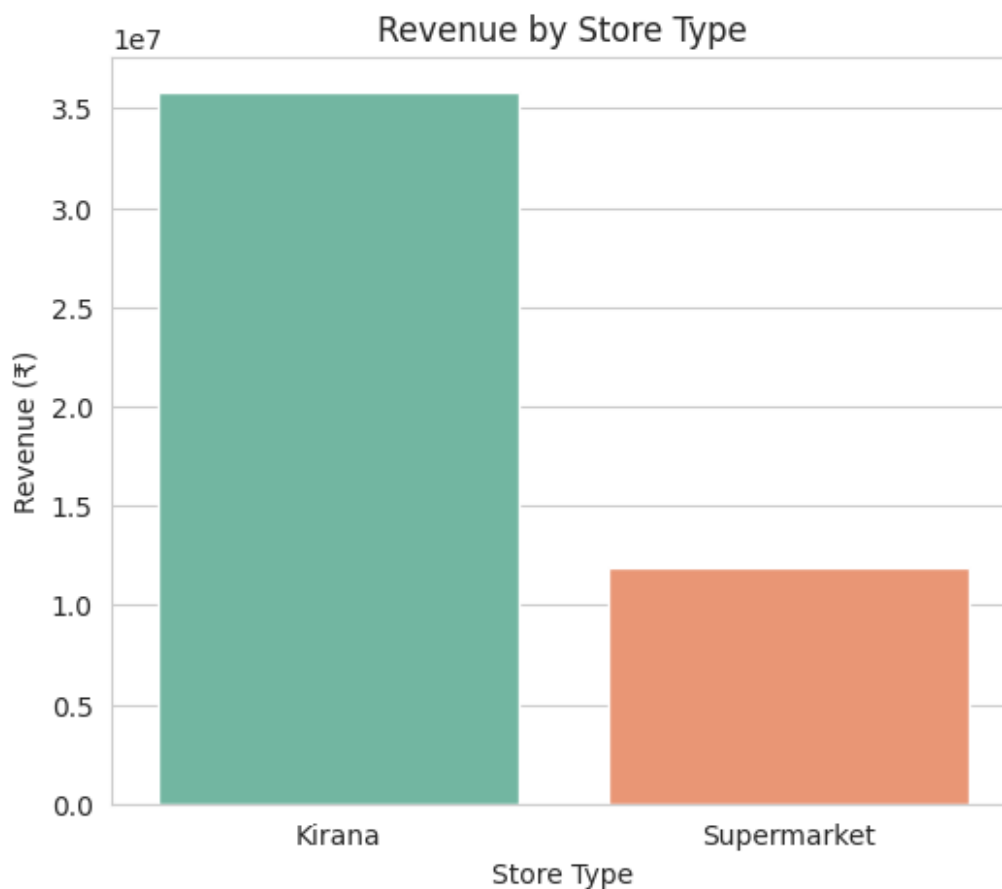
plt.show()

store_sales

/tmp/ipykernel_1610/2946103212.py:5: FutureWarning:
Passing `palette` without assigning `hue` is deprecated and will be
removed in v0.14.0. Assign the `x` variable to `hue` and set
`legend=False` for the same effect.

sns.barplot(

```



```

store_type
Kirana      35807170

```

```
Supermarket    11846780  
Name: total_amount, dtype: int64
```

```
category_sales = (  
    df.groupby('category')['total_amount']  
      .sum()  
      .sort_values(ascending=False)  
)
```

```
plt.figure(figsize=(12,6))
```

```
sns.barplot(  
    x=category_sales.values,  
    y=category_sales.index,  
    palette='rocket'  
)
```

```
plt.title("Revenue by Product Category")  
plt.xlabel("Revenue (₹)")  
plt.ylabel("Category")
```

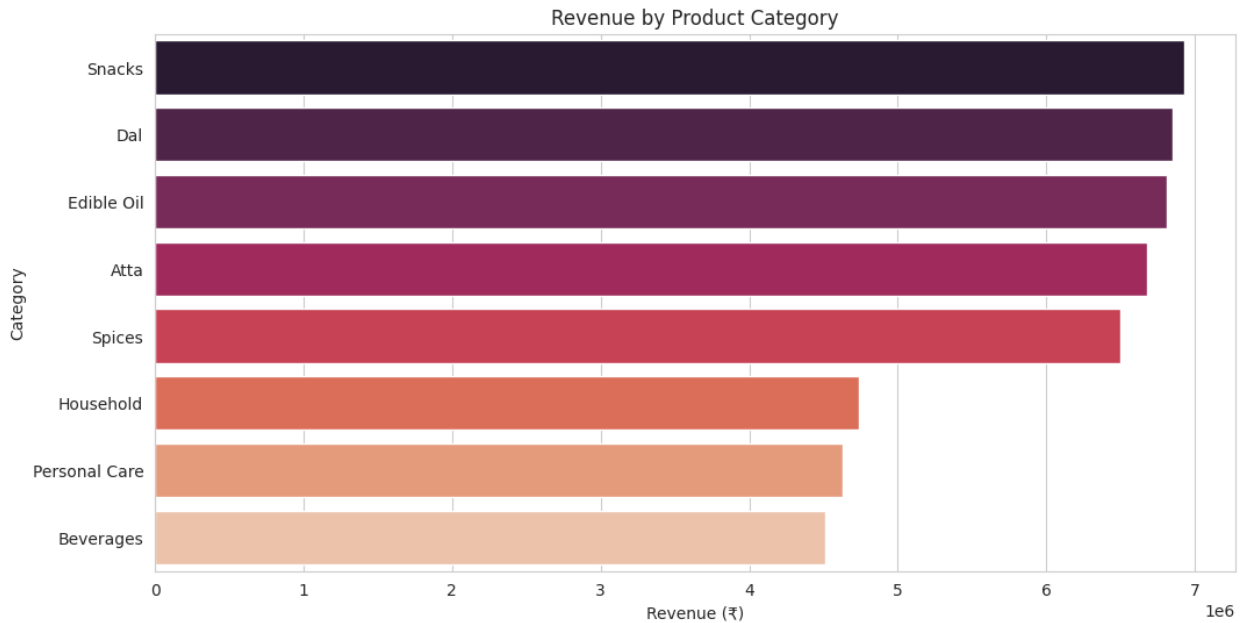
```
plt.show()
```

```
category_sales
```

```
/tmp/ipykernel_1610/1356488176.py:9: FutureWarning:
```

```
Passing `palette` without assigning `hue` is deprecated and will be  
removed in v0.14.0. Assign the `y` variable to `hue` and set  
`legend=False` for the same effect.
```

```
sns.barplot(  
    x=category_sales.values,  
    y=category_sales.index,  
    hue=category_sales.index,  
    legend=False,  
    palette='rocket'
```



```
category
Snacks      6926430
Dal         6851770
Edible Oil  6811800
Atta        6678890
Spices      6497960
Household   4738970
Personal Care 4632780
Beverages   4515350
Name: total_amount, dtype: int64

channel_sales = df.groupby('channel')['total_amount'].sum()

plt.figure(figsize=(6,5))

sns.barplot(
    x=channel_sales.index,
    y=channel_sales.values,
    palette='Pastel2'
)

plt.title("Revenue by Sales Channel")
plt.xlabel("Channel")
plt.ylabel("Revenue (₹)")

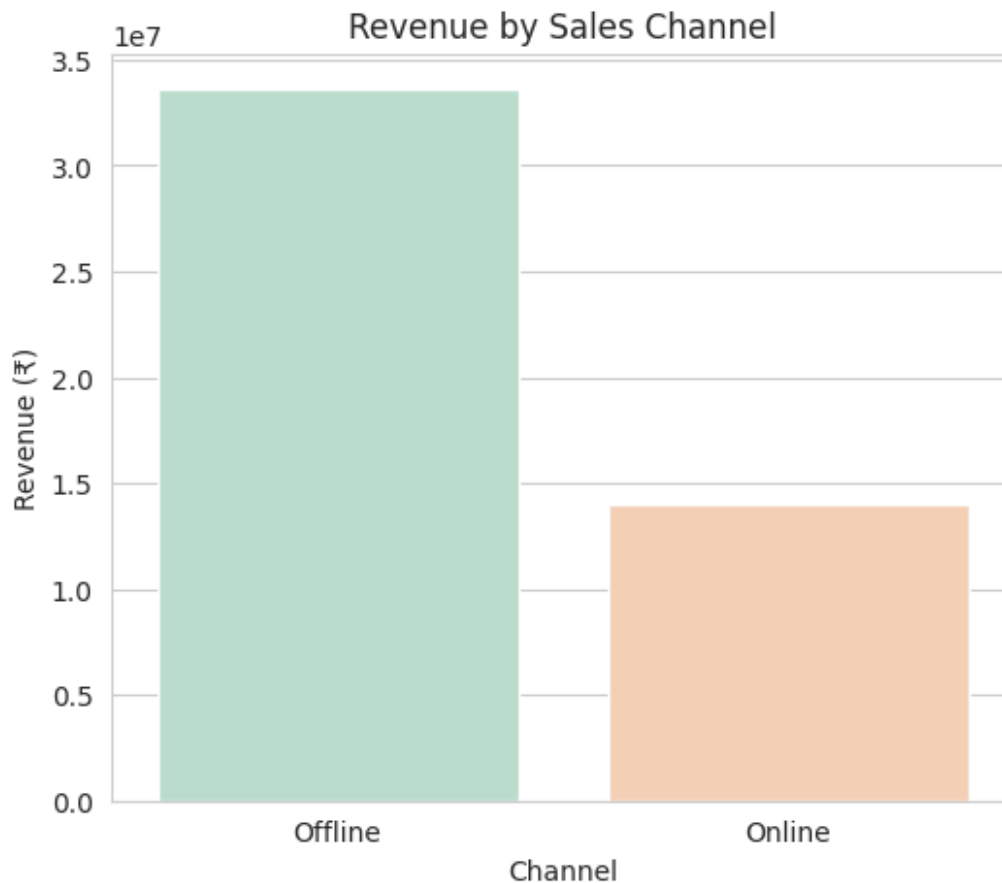
plt.show()

channel_sales

/tmp/ipykernel_1610/1247835547.py:5: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(
```



```
channel
Offline    33612590
Online     14041360
Name: total_amount, dtype: int64

df['Month_Name'] = df['order_date'].dt.month_name()

monthly_sales = (
    df.groupby('Month_Name')['total_amount']
      .sum()
      .reindex([
          'January', 'February', 'March', 'April',
          'May', 'June', 'July', 'August',
          'September', 'October', 'November', 'December'
      ])
)
```

```
plt.figure(figsize=(12,6))

sns.lineplot(
    x=monthly_sales.index,
    y=monthly_sales.values,
    marker='o',
    linewidth=3,
    color='blue'
)

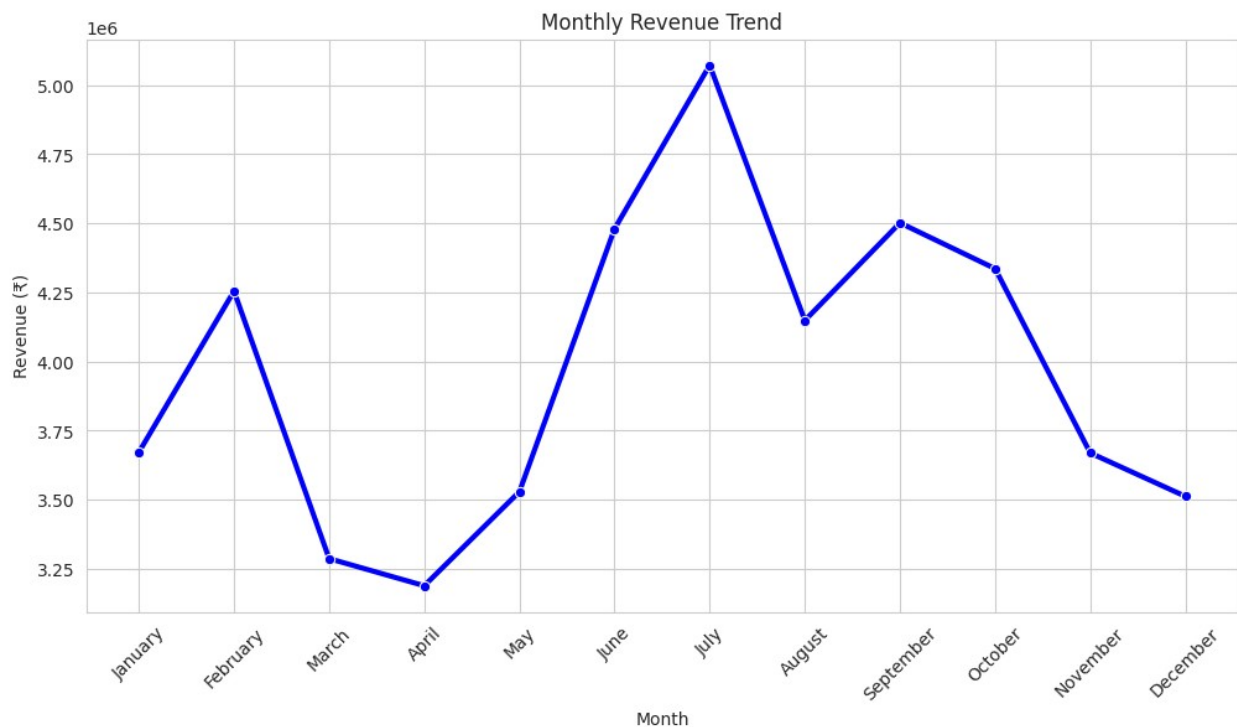
plt.xticks(rotation=45)

plt.title("Monthly Revenue Trend")
plt.xlabel("Month")
plt.ylabel("Revenue (₹)")

plt.grid(True)

plt.show()

monthly_sales
```



Month_Name	
January	3671930
February	4255000
March	3287310

```
April      3189090
May        3529240
June       4480260
July       5072420
August     4148900
September  4501790
October    4336160
November   3669390
December   3512460
Name: total_amount, dtype: int64
```

```
monthly_orders = (
    df.groupby('Month_Name')
      .size()
      .reindex([
          'January', 'February', 'March', 'April',
          'May', 'June', 'July', 'August',
          'September', 'October', 'November', 'December'
      ])
)
```

```
plt.figure(figsize=(12,6))
```

```
sns.lineplot(
    x=monthly_orders.index,
    y=monthly_orders.values,
    marker='o',
    linewidth=3,
    color='green'
)
```

```
plt.xticks(rotation=45)
```

```
plt.title("Monthly Order Trend")
```

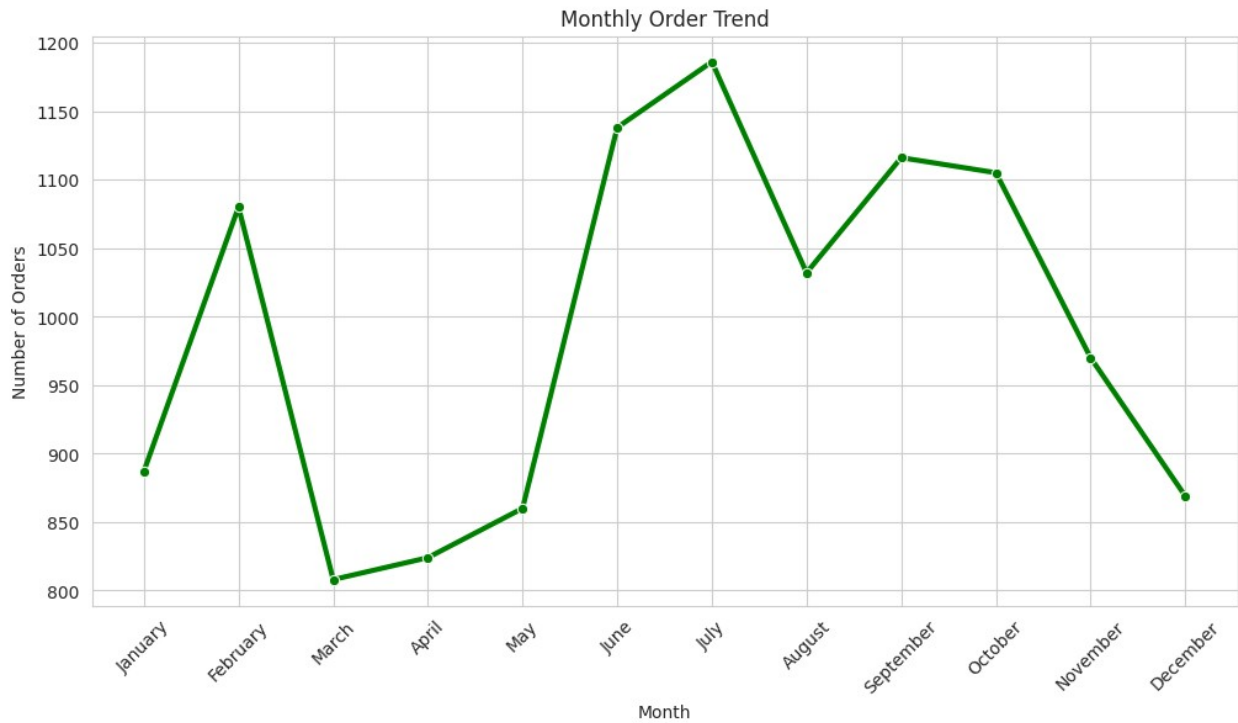
```
plt.xlabel("Month")
```

```
plt.ylabel("Number of Orders")
```

```
plt.grid(True)
```

```
plt.show()
```

```
monthly_orders
```

```
Month_Name
January      887
February    1080
March        808
April        824
May          860
June        1138
July         1186
August       1032
September    1116
October      1105
November     970
December     869
dtype: int64

df['Weekday'] = df['order_date'].dt.day_name()
weekday_orders = (
    df.groupby('Weekday')
      .size()
      .reindex(
        ['Monday',
         'Tuesday',
         'Wednesday',
         'Thursday',
         'Friday',
         'Saturday',
         'Sunday']
      )
)
```

```

    1)
)
plt.figure(figsize=(10,5))
sns.barplot(
    x=weekday_orders.index,
    y=weekday_orders.values,
    palette='Set2'
)

plt.title("Orders by Weekday")
plt.xlabel("Weekday")
plt.ylabel("Number of Orders")

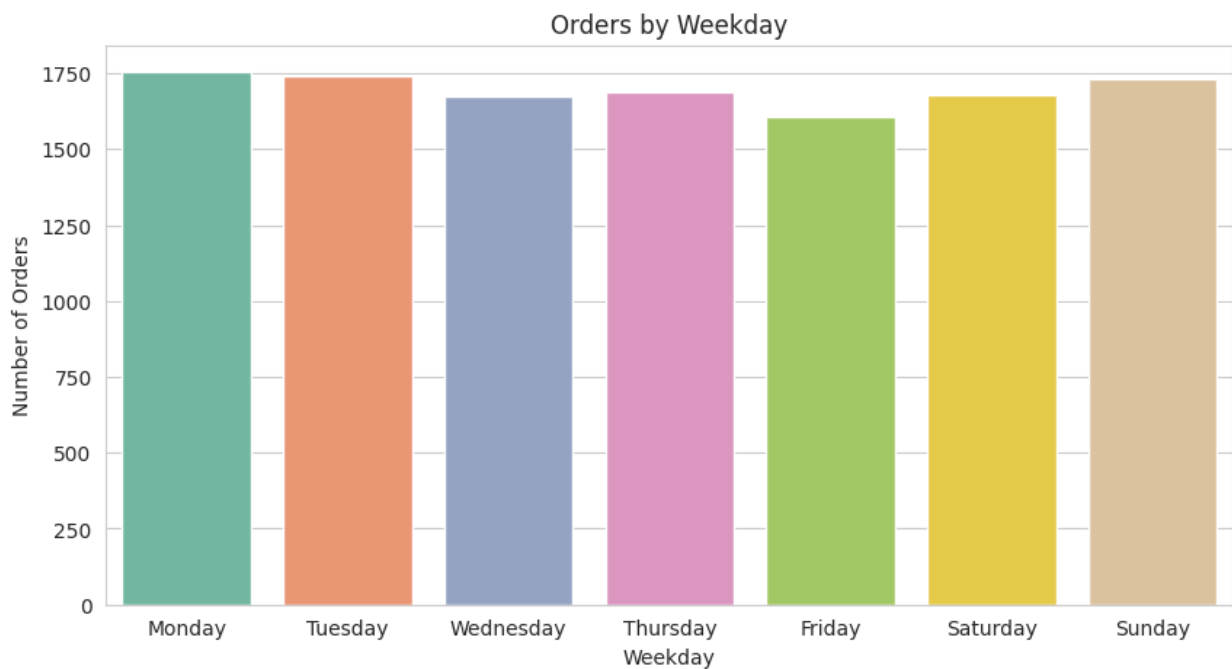
plt.show()

weekday_orders

/tmp/ipykernel_1610/3753563962.py:18: FutureWarning:
Passing `palette` without assigning `hue` is deprecated and will be
removed in v0.14.0. Assign the `x` variable to `hue` and set
`legend=False` for the same effect.

sns.barplot(

```



Weekday	Number of Orders
Monday	1755

```
Tuesday      1740
Wednesday    1673
Thursday     1689
Friday       1608
Saturday     1680
Sunday       1730
dtype: int64
```

```
weekday_sales = (
    df.groupby('Weekday')['total_amount']
      .sum()
      .reindex([
          'Monday',
          'Tuesday',
          'Wednesday',
          'Thursday',
          'Friday',
          'Saturday',
          'Sunday'
      ])
)
```

```
plt.figure(figsize=(10,5))
```

```
sns.barplot(
    x=weekday_sales.index,
    y=weekday_sales.values,
    palette='coolwarm'
)
```

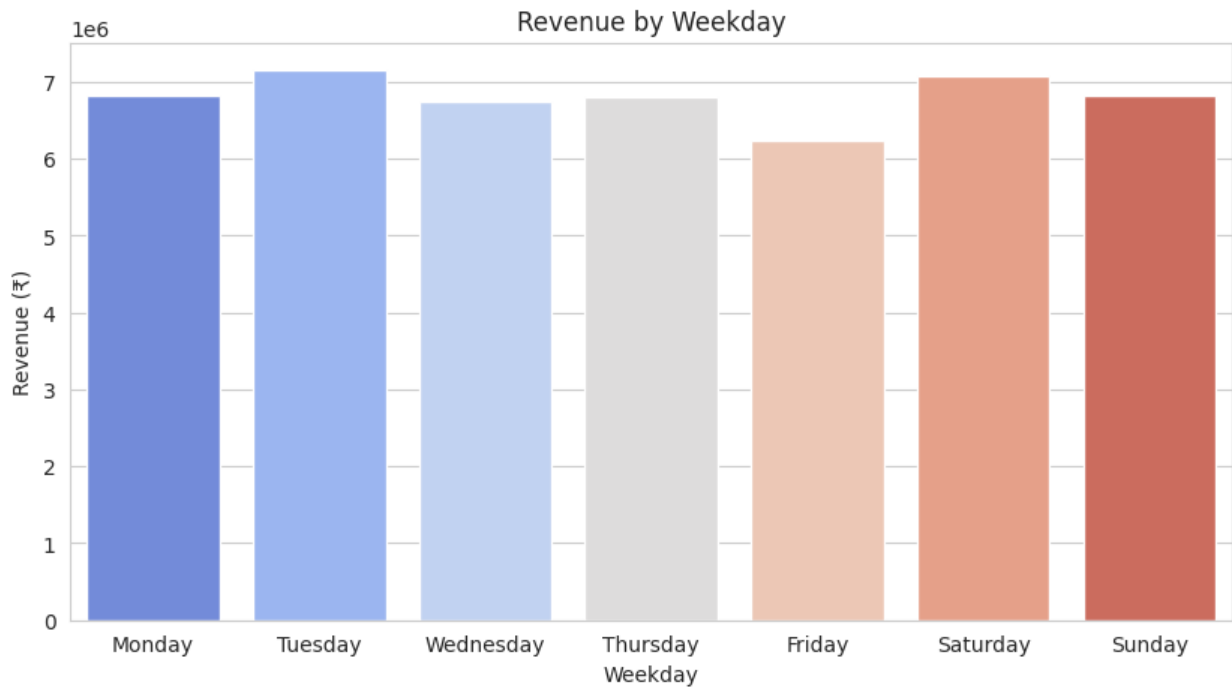
```
plt.title("Revenue by Weekday")
plt.xlabel("Weekday")
plt.ylabel("Revenue (₹)")
```

```
plt.show()
```

```
/tmp/ipykernel_1610/275398950.py:17: FutureWarning:
```

```
Passing `palette` without assigning `hue` is deprecated and will be
removed in v0.14.0. Assign the `x` variable to `hue` and set
`legend=False` for the same effect.
```

```
sns.barplot(
```



```
df['Week_Number'] = df['order_date'].dt.isocalendar().week
weekly_orders = (
    df.groupby('Week_Number')
      .size()
)

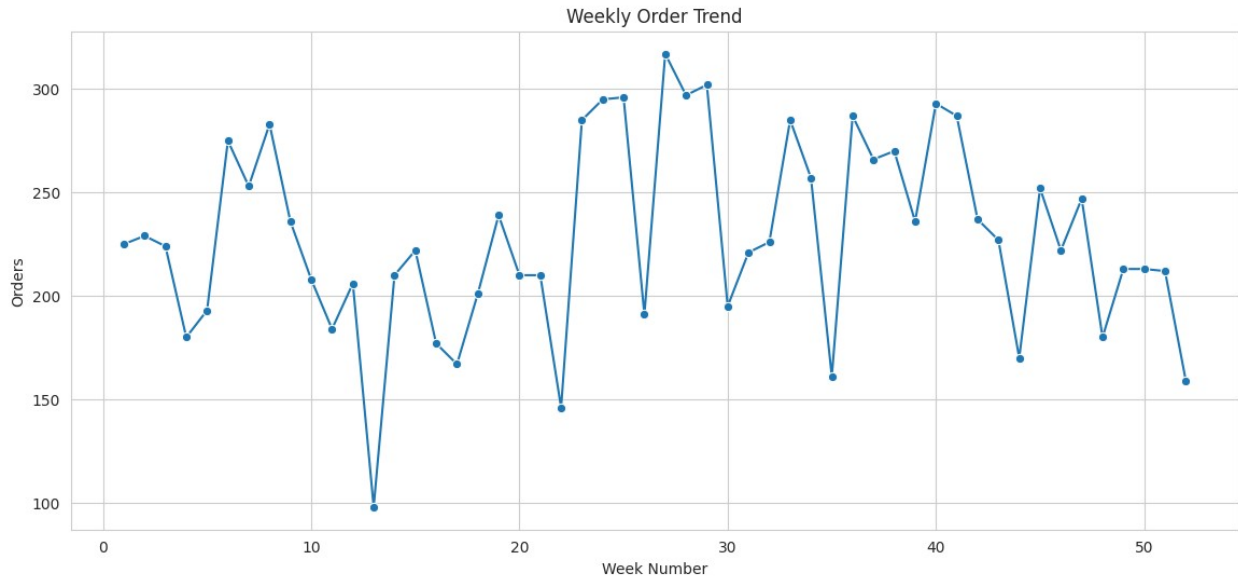
plt.figure(figsize=(14,6))

sns.lineplot(
    x=weekly_orders.index,
    y=weekly_orders.values,
    marker='o'
)

plt.title("Weekly Order Trend")
plt.xlabel("Week Number")
plt.ylabel("Orders")

plt.grid(True)

plt.show()
```



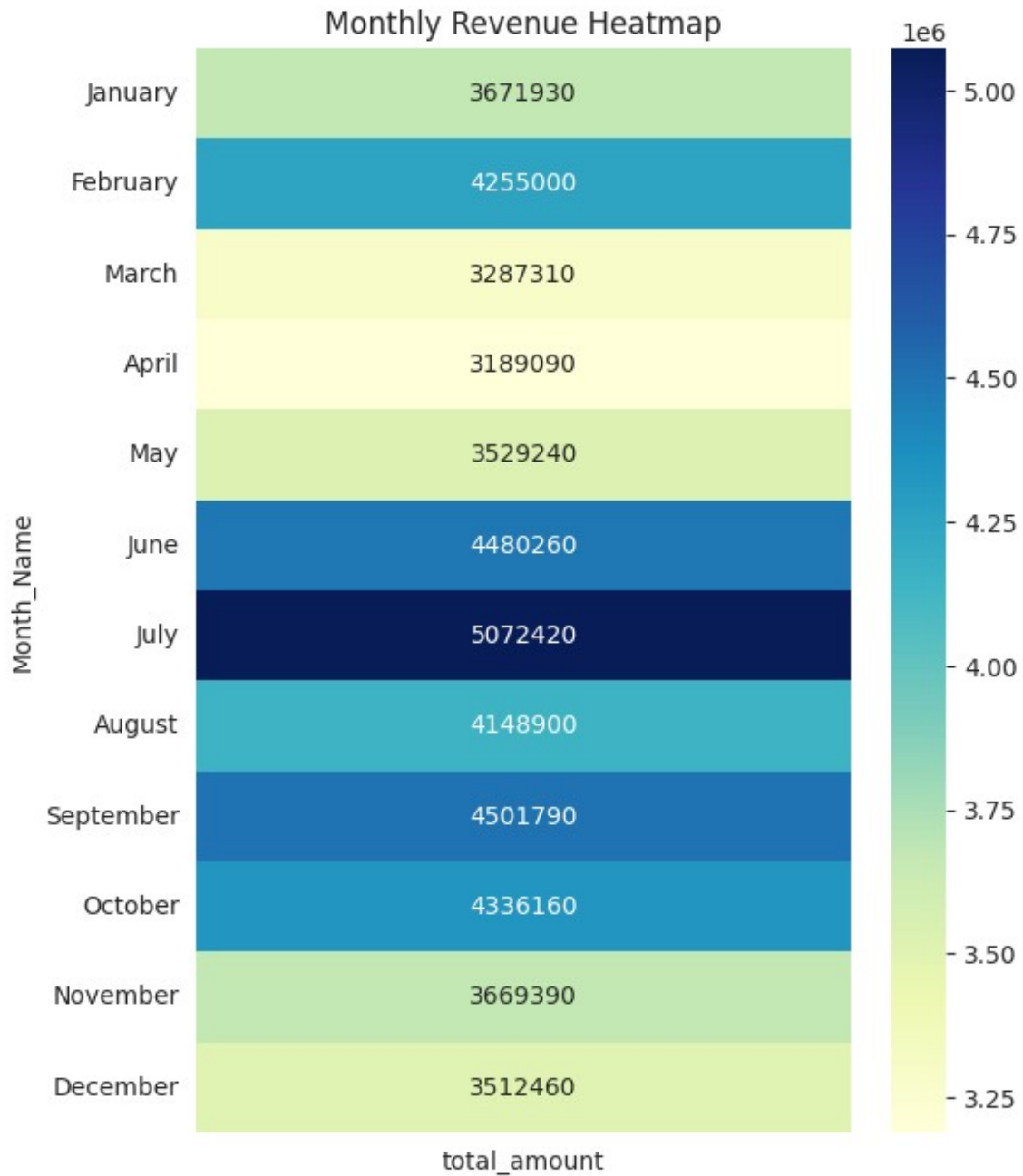
```
monthly_heatmap = (
    df.groupby('Month_Name')['total_amount']
      .sum()
      .reindex([
          'January', 'February', 'March', 'April',
          'May', 'June', 'July', 'August',
          'September', 'October', 'November', 'December'
      ])
      .to_frame()
)

plt.figure(figsize=(6,8))

sns.heatmap(
    monthly_heatmap,
    annot=True,
    fmt=".0f",
    cmap="YlGnBu"
)

plt.title("Monthly Revenue Heatmap")

plt.show()
```



```
# Overall Return Rate
total_orders = len(df)
returned_orders = len(df[df['Return_Status'] == 'Returned'])
return_rate = (returned_orders / total_orders) * 100
print("Total Orders :", total_orders)
```

```

print("Returned Orders :", returned_orders)
print("Return Rate : {:.2f}%".format(return_rate))

# Return Rate by Category

category_return = (
    df.groupby('category')['Return_Status']
      .apply(lambda x: (x == 'Returned').mean() * 100)
      .sort_values(ascending=False)
)

plt.figure(figsize=(12,6))

sns.barplot(
    x=category_return.values,
    y=category_return.index,
    palette='Reds_r'
)

plt.title("Return Rate by Product Category")
plt.xlabel("Return Rate (%)")
plt.ylabel("Category")

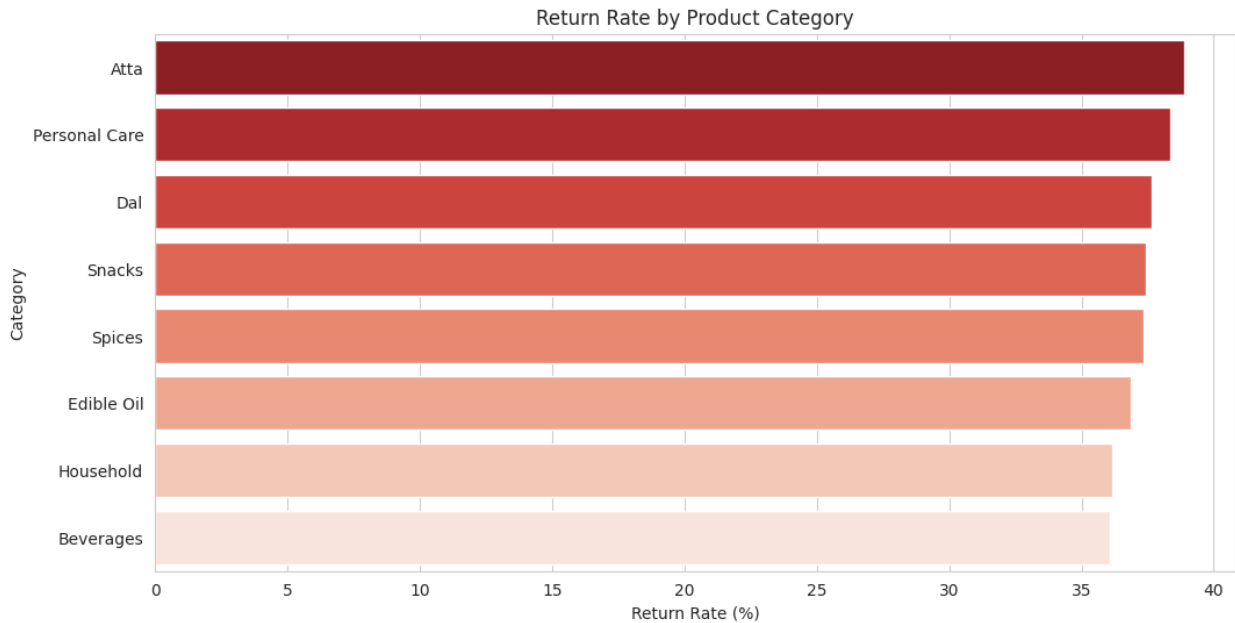
plt.show()

category_return

/tmp/ipykernel_1269/3114325136.py:11: FutureWarning:
Passing `palette` without assigning `hue` is deprecated and will be
removed in v0.14.0. Assign the `y` variable to `hue` and set
`legend=False` for the same effect.

sns.barplot(

```



```
category
Atta      38.872404
Personal Care  38.352524
Dal       37.672811
Snacks    37.434095
Spices    37.341390
Edible Oil 36.879857
Household 36.148649
Beverages 36.075949
Name: Return_Status, dtype: float64

city_return = (
    df.groupby('city')['Return_Status']
      .apply(lambda x: (x == 'Returned').mean() * 100)
      .sort_values(ascending=False)
)

plt.figure(figsize=(10,6))

sns.barplot(
    x=city_return.values,
    y=city_return.index,
    palette='Oranges_r'
)

plt.title("Return Rate by City")
plt.xlabel("Return Rate (%)")
plt.ylabel("City")

plt.show()
```

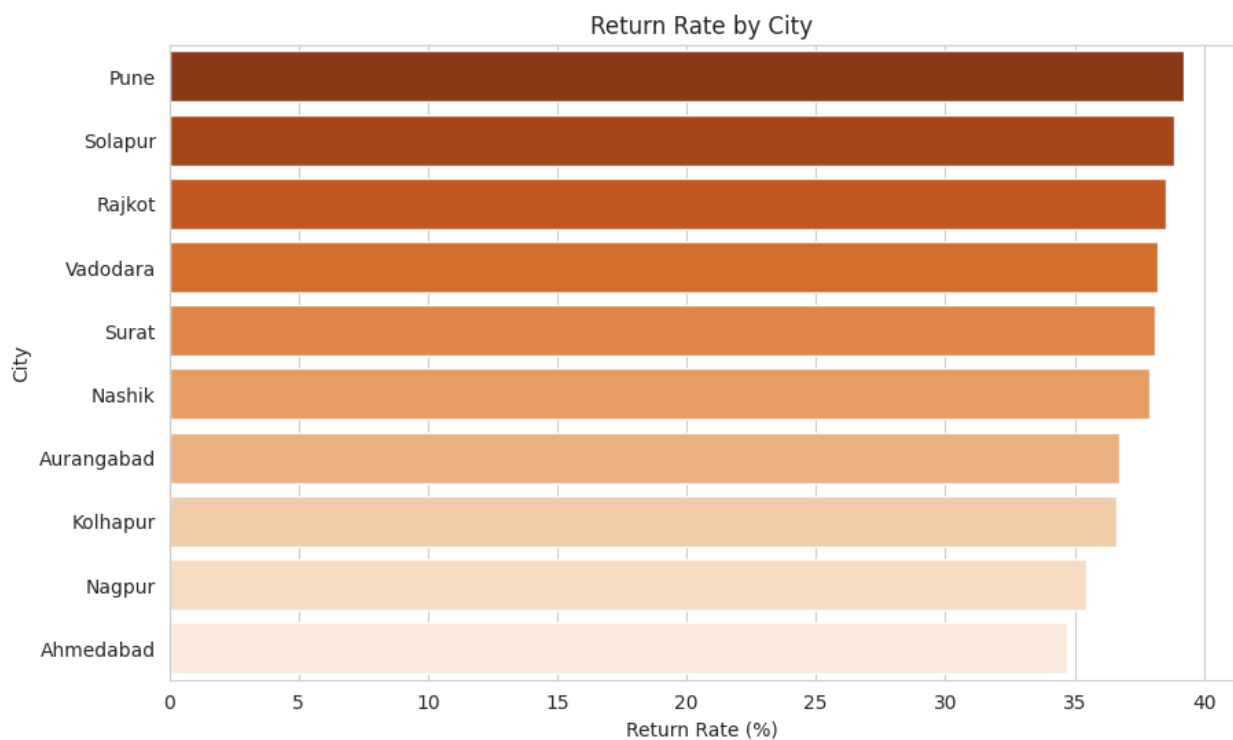


```
city_return
```

```
/tmp/ipykernel_1610/3641736495.py:9: FutureWarning:
```

```
Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.
```

```
sns.barplot(
```



```
city
Pune      39.201452
Solapur   38.833333
Rajkot    38.521066
Vadodara  38.196721
Surat     38.119658
Nashik    37.869822
Aurangabad 36.747967
Kolhapur  36.602871
Nagpur     35.454545
Ahmedabad 34.733158
Name: Return_Status, dtype: float64
```

```
channel_return = (
    df.groupby('channel')['Return_Status']
    .apply(lambda x: (x == 'Returned').mean() * 100)
```

```
)  
plt.figure(figsize=(6,5))  
sns.barplot(  
    x=channel_return.index,  
    y=channel_return.values,  
    palette='Set2'  
)  
  
plt.title("Return Rate by Sales Channel")  
plt.xlabel("Channel")  
plt.ylabel("Return Rate (%)")  
  
plt.show()  
  
channel_return  
  
/tmp/ipykernel_1610/923096016.py:8: FutureWarning:  
Passing `palette` without assigning `hue` is deprecated and will be  
removed in v0.14.0. Assign the `x` variable to `hue` and set  
`legend=False` for the same effect.  
  
sns.barplot(
```



```
channel
Offline    37.687959
Online     36.777092
Name: Return_Status, dtype: float64

store_return = (
    df.groupby('store_type')['Return_Status']
      .apply(lambda x: (x == 'Returned').mean() * 100)
)

plt.figure(figsize=(6,5))

sns.barplot(
    x=store_return.index,
    y=store_return.values,
    palette='coolwarm'
)

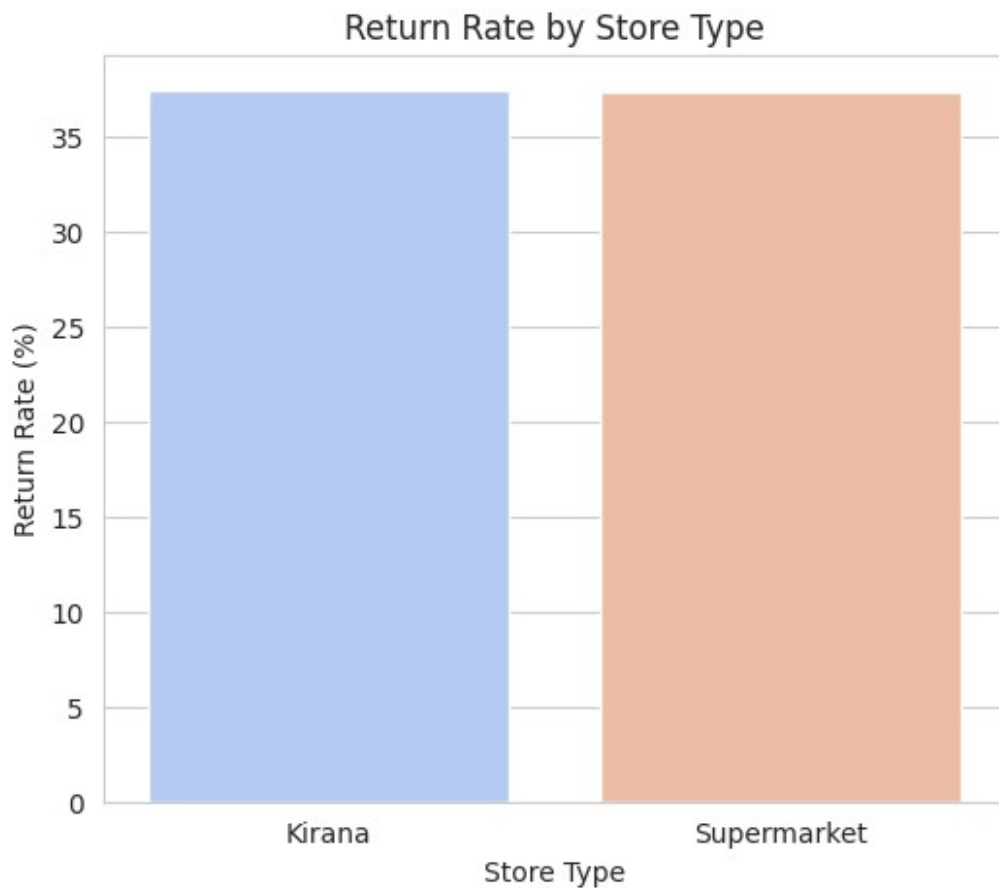
plt.title("Return Rate by Store Type")
plt.xlabel("Store Type")
plt.ylabel("Return Rate (%)")
```

```
plt.show()

store_return

/tmp/ipykernel_1269/1025898200.py:8: FutureWarning:
Passing `palette` without assigning `hue` is deprecated and will be
removed in v0.14.0. Assign the `x` variable to `hue` and set
`legend=False` for the same effect.

sns.barplot(
```



```
store_type
Kirana      37.432675
Supermarket  37.360783
Name: Return_Status, dtype: float64

plt.figure(figsize=(8,5))

sns.countplot(
    data=df,
    y='return_reason',
```

```

    order=df['return_reason'].value_counts().index,
    palette='magma'
)

plt.title("Reasons for Product Returns")
plt.xlabel("Number of Returns")
plt.ylabel("Return Reason")

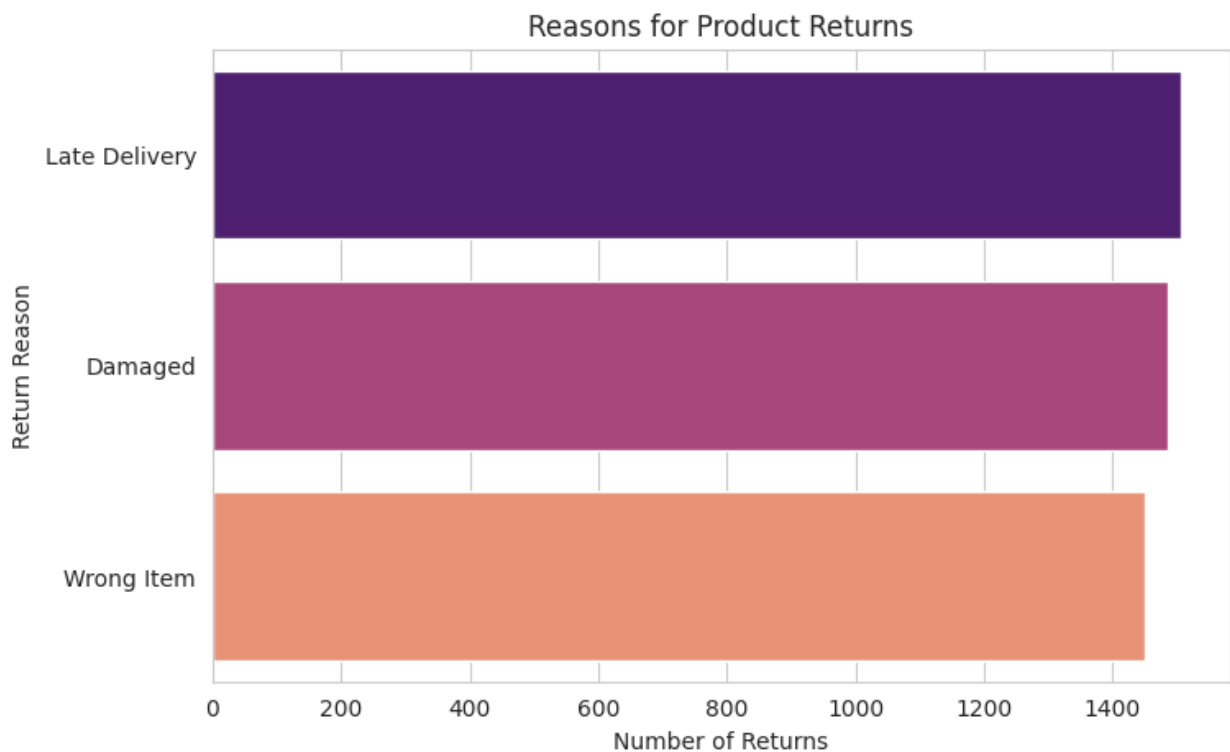
plt.show()

df['return_reason'].value_counts()

/tmp/ipykernel_1610/927119098.py:3: FutureWarning:
Passing `palette` without assigning `hue` is deprecated and will be
removed in v0.14.0. Assign the `y` variable to `hue` and set
`legend=False` for the same effect.

sns.countplot(

```



```

return_reason
Late Delivery    1507
Damaged          1486
Wrong Item       1450
Name: count, dtype: int64

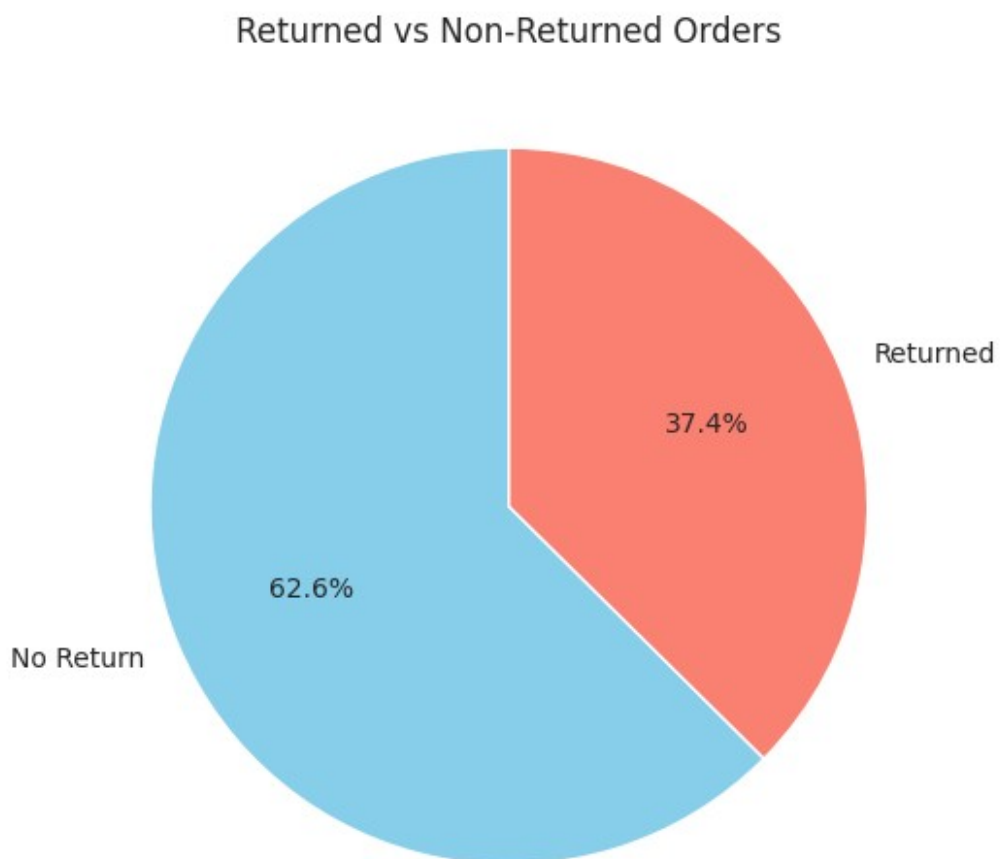
```

```
plt.figure(figsize=(6,6))

df['Return_Status'].value_counts().plot(
    kind='pie',
    autopct='%1.1f%%',
    startangle=90,
    colors=['skyblue', 'salmon']
)

plt.title("Returned vs Non-Returned Orders")
plt.ylabel("")

plt.show()
```



```
# Encode categorical variables

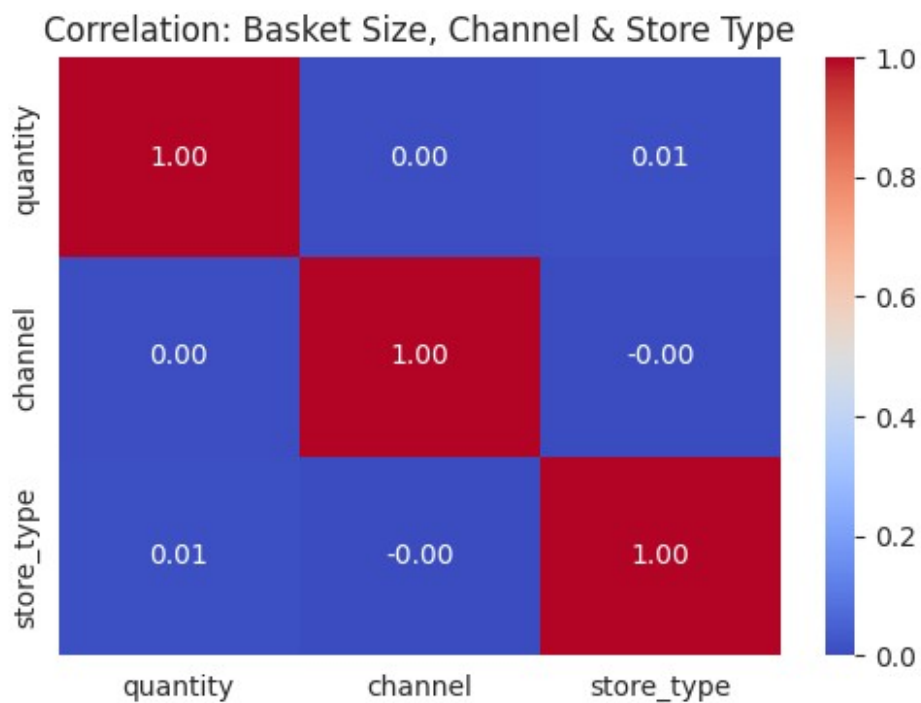
corr_df = df.copy()

corr_df['channel'] = corr_df['channel'].map({
    'Offline': 0,
```

```

    'Online': 1
})
corr_df['store_type'] = corr_df['store_type'].map({
    'Kirana': 0,
    'Supermarket': 1
})
# Correlation Matrix
corr = corr_df[['quantity', 'channel', 'store_type']].corr()
plt.figure(figsize=(6,4))
sns.heatmap(
    corr,
    annot=True,
    cmap='coolwarm',
    fmt='.2f'
)
plt.title("Correlation: Basket Size, Channel & Store Type")
plt.show()
corr

```



```
{
  "summary": "{\n  \"name\": \"corr\",\n  \"rows\": 3,\n  \"fields\": [\n    {\n      \"column\": \"quantity\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0.5733495007459177,\n        \"min\": 0.003688500129301042,\n        \"max\": 1.0,\n        \"num_unique_values\": 3,\n        \"samples\": [\n          0.003688500129301042,\n          0.010202615908400381,\n          1.0\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"channel\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0.5763840304875709,\n        \"min\": -0.00032922416745174613,\n        \"max\": 1.0,\n        \"num_unique_values\": 3,\n        \"samples\": [\n          0.003688500129301042,\n          0.00032922416745174613,\n          1.0\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    }\n  ]\n}\n",
  "type": "dataframe",
  "variable_name": "corr"
}
```